CHAPTER - 5

Computational Complexity Theory

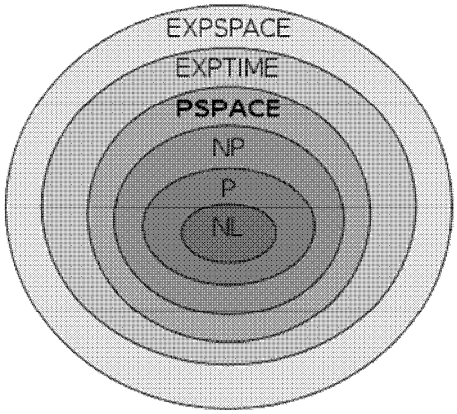
5.1 Introduction

Complexity theory considers not only whether a problem can be solved at all on a computer (as computability theory), but also how efficiently the problem can be solved. Thus, the complexity theory classifies problems according to their degree of "difficulty". Two major aspects are considered: time complexity and space complexity, which are respectively how many steps does it take to perform a computation, and how much memory is required to perform that computation. Although the time and space complexity are dependent of input problem but it is more complexity issue, so computer scientists have adopted Big O notation that compare the asymptotic behavior of large problems.

Complexity classes

Complexity class	Model of computation	Resource constraint
P	Deterministic Turing machine	Time poly(n)
EXPTIME	Deterministic Turing machine	Time $2^{poly(n)}$
NP	Non-deterministic Turing machine	Time poly(n)
NEXPTIME	Non-deterministic Turing machine	Time $2^{poly(n)}$
L	Deterministic Turing machine	Space O(log n)
PSPACE	Deterministic Turing machine	Space poly(n)
EXPSPACE	Deterministic Turing machine	Space $2^{poly(n)}$
NL	Non-deterministic Turing machine	Space O(log n)
NPSPACE	Non-deterministic Turing machine	Space poly(n)
NEXPSPACE	Non-deterministic Turing machine	Space $2^{poly(n)}$

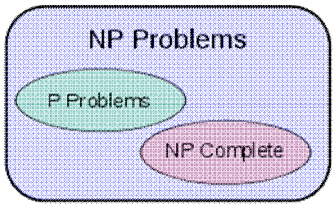
5.2 Class P and NP problems



a. **P = Polynomial** (*Problems that solve by deterministic algorithm i.e. by TM that halts*)
b. **NP = Non-deterministic polynomial** (*Problems that solve by non-deterministic algorithm i.e. by NTM*)

- The ***time complexity,*** **T(n)** of Turing machine (M) is the maximum moves made by M in passing any input string of length 'n' i.e. *M halts after making at most T(n) moves.*
- For defining class P and class NP, we require the time complexity of a Turing machine.
- A language L is said to be in class P if there exists a deterministic Turing machine M such that M is of time complexity P(n) for some polynomial P and M accept L. E.g. Kruskal's Algorithm
- A language L is said to be in class NP if there exists a non-deterministic Turing machine M such that M is of time complexity P(n) for some polynomial P and M accept L. E.g. Traveling sales man problem

P versus NP problem



- The complexity class P is often seen as a mathematical abstraction modeling those computational tasks that admit an efficient algorithm.
- The complexity class NP, on the other hand, contains many problems that people would like to solve efficiently, but for

which no efficient algorithm is known. Diagram of complexity classes provided that $P \neq NP$.

- A deterministic machine, at each point in time, executes an instruction. Depending on the outcome of executing the instruction, it then executes some next instruction, which is unique. A non-deterministic machine on the other hand has a choice of next steps. It is free to choose any that it wishes. For example, it can always choose a next step that leads to the best solution for the problem. A non-deterministic machine thus has the power of extremely good, optimal guessing.

The **P** versus **NP** problem is to determine whether every language accepted by some nondeterministic algorithm in polynomial time is also accepted by some (deterministic) algorithm in polynomial time. To define the problem precisely it is necessary to give a formal model of a computer. The standard computer model in computability theory is the Turing machine, introduced by Alan Turing in 1936. Although the model was introduced before physical computers were built, it nevertheless continues to be accepted as the proper computer model for the purpose of defining the notion of *computable function*.

Informally the class **P** is the class of decision problems solvable by some algorithm within a number of steps bounded by some fixed polynomial in the length of the input. Turing was not concerned with the efficiency of his machines, but rather his concern was whether they can simulate arbitrary algorithms given sufficient time. However it turns out Turing machines can generally simulate more efficient computer models (for example machines equipped with many tapes or an unbounded random access memory) by at most squaring or cubing the computation time. Thus **P** is a robust class, and has equivalent definitions over a large class of computer models.

NP-completeness

NP-complete is a subset of NP, the set of all decision problems whose solutions can be verified in polynomial time; *NP* may be equivalently defined as the set of decision problems that can be solved in polynomial time on a nondeterministic Turing machine.

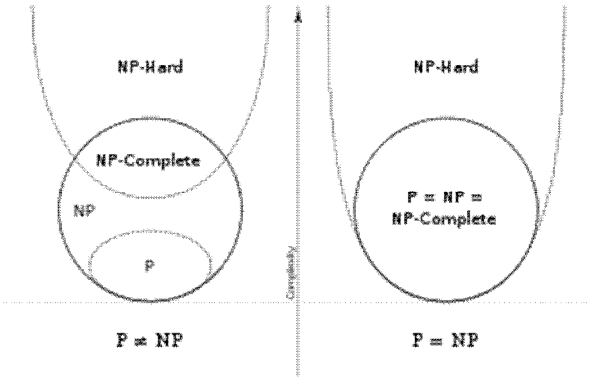
A decision problem C is NP-complete if:

1. C is in NP, and
2. Every problem in NP is reducible to C in polynomial time.

NP-hard problems

Some problem that proves the condition – (2) of NP-completeness problem but not the condition – (1) is called the NP-hard problem. They are intractable problems.

Euler diagram for P, NP, NP-complete, and NP-hard set of problems can be seen here.



Note: Here we define the language as problem in case of all above complexity class problems.